

Google Chrome SSE DNS Rebinding

This vulnerability is a security flaw that could enable DNS rebinding attacks when SSE is used. The issue stems from the failure to properly restrict the `Access-Control-Allow-Origin: *` header, which, under certain circumstances, allows connections from unauthorized origins.

SOP/CORS

In fact, the `*` wildcard can be dangerous depending on the context. The `*` wildcard may be normal for public resources, but it is considered dangerous if it exposes sensitive resources to all origins. This vulnerability is called a CORS misconfiguration.



A CORS misconfiguration occurs when the server fails to properly restrict which origins' requests—resulting in **unauthorized sites being able to access backend resources**

There is also the concept of **SOP (Same-Origin Policy)**, which is a browser security rule that restricts access to resources from different origins. **CORS, on the other hand**, is a mechanism that allows the server to grant permission for access to specific different origins. BTW, in SOP, **subdomains are considered different origins**.

The fact that subdomains are under the same "main domain" does not automatically make them same-origin from an SOP perspective. When CORS is needed, the server can grant explicit permission to a specific origin, such as

`https://app.example.com`.

Technical Details

In `internal/server/mcp.go:370`, the `sseHandler` manually sets the header:

```
w.Header().Set("Access-Control-Allow-Origin", "*")
```

This hardcoded value overrides the global CORS policy, allowing any website to establish a connection to the toolbox, hijack session IDs, and execute arbitrary tools on behalf of the user.

DNS Rebinding

DNS (Domain Name System) is the system that translates domain names into IP addresses. When a user types `example.com` into a browser, the browser asks the DNS server which IP address corresponds to that domain. The DNS returns a response, such as `93.184.216.34`, and the browser sends a request to the server at that IP address.

In a **DNS rebinding** attack, the attacker alters the domain's DNS response after a certain period of time. For example, initially, `the domain` is directed to the `attacker's` server's IP address, and the browser loads JavaScript from that site; later, the DNS redirects the same domain to the victim's local or internal server's IP address. Thus, the attacker attempts to send requests to the internal server via the JavaScript running in the browser and retrieve its data.

The main reason behind this process is a low TTL. In other words, using a low TTL in DNS rebinding causes the DNS response to expire quickly, creating conditions for the domain to be subsequently redirected to another address—such as an internal IP address.

İlk DNS cavabı:
attacker.com → Attacker IP

TTL bitir

Yeni DNS cavabı:
attacker.com → 127.0.0.1 / Internal IP

In the first DNS response, the attacker provides the attacker's IP address so that the user's browser **will** first **load the malicious JavaScript** from the attacker's server.

In the second DNS response, the browser has already begun executing the attacker's JavaScript, and subsequent requests made by that JavaScript may be directed to a different IP address.

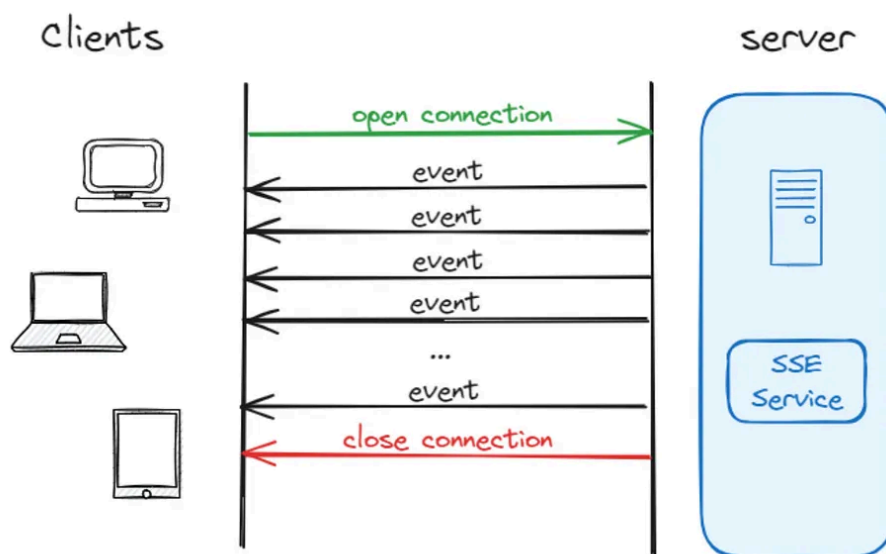
The heart of the attack

During **DNS rebinding**, a domain can first be redirected to the attacker's server and then to an internal server. Since **SSE** allows the browser to maintain a long-lived HTTP connection with that server, if an application using an SSE connection in Chrome does not perform proper origin/host checks, these mechanisms combined can create a security issue.



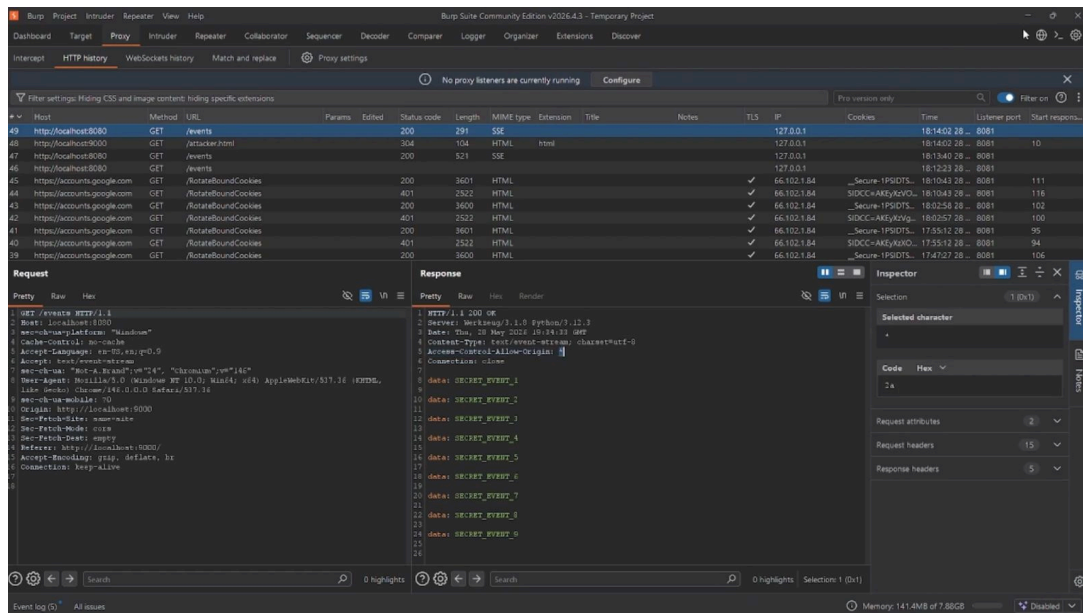
SSE (Server-Sent Events) is a technology that allows the server to **send real-time data to the client in a single direction (server → client)**.

The key feature of SSE is the dynamic delivery of data. The connection remains open, and the server **sends new events to the browser as they occur**; the browser does not send a separate request each time.



SSE yalnız serverdən client-ə kommunikasiya üçün nəzərdə tutulub, yəni client-dən serverə data stream etmək mümkün deyil.

Example



Burada `Access-Control-Allow-Origin: *` problemi qeyd olunub və hər dəfə `SECRET_EVENT` göndərilir. (`SECRET_EVENT_[NUM]` sadəcə praktika üçün yazılmışdır).

Fix

1. **Host validation** — the server should only accept expected hostnames. In other words, relying solely on IP addresses is not sufficient.
2. **Origin validation** — `The origin` can be verified in **SSE** and other browser requests.
3. Properly configuring CORS — only the necessary origins should be added to the allowlist.
4. **Authentication and authorization** — Authentication and authorization must be implemented to ensure that no client accessing the **endpoint** can read the data.

References

- <https://nvd.nist.gov/vuln/detail/cve-2026-9739>
- <https://github.com/googleapis/mcp-toolbox/issues//3053>
- <https://github.com/googleapis/mcp-toolbox/pull/3054>
- <https://kennethnnaue.medium.com/introduction-to-server-sent-events-sse-d0b7590503a5>